ASSAR Insurance Pricing & Information Engine

Technical Report: Architecture, Data Design, Retrieval, and Audit

This report documents the ASSAR pricing-and-information system end to end: what it does, how it is built, the design choices behind the data and retrieval layers, and a full audit table mapping every SQL table back to its source table and page in the manual. The source throughout is the Association of Insurers of Rwanda (ASSAR) Approved General Business Pricing Manual for the Rwandan Insurance Industry, Version 3, effective 25 January 2021 (81 pages).

---

1. Introduction

Insurance pricing manuals are written for people to read, not for machines to query. The ASSAR manual interleaves dense numeric rate tables, a 104-row fire grid, transit and marine commodity grids, liability and engineering rates, bonds, political-violence rates, and registers of large risks, with paragraphs of underwriting prose: definitions, conditions, warranties, exclusions, and guidance. A reader who wants a single number has to find the right page; a reader who wants to understand a clause has to read several.

This system turns that manual into something a client or underwriter can query in plain language. It answers two distinct kinds of question. The first is quantitative and exact: what minimum rate applies to a risk, what multiplier applies for an indemnity period, what the mandatory excess is, what premium a given cover costs. The second is qualitative: what a term means, what is excluded, which warranty must be incorporated. The system is deliberately built so that the exact numbers are served exactly and deterministically, while the open-ended questions are answered fluently and with citations.

The result is a conversational assistant with three faces: a chat that holds a multi-turn conversation, a structured quote calculator, and a browsable database. All of it is grounded in the manual, and every premium is computed by tested code rather than guessed by a language model.

2. System Architecture

The architecture follows one principle: numbers and prose are stored and retrieved differently, and a language model orchestrates but never computes.

A free-text question first reaches a Manager/router that classifies it. If it is a pricing or number request, it is sent to the Quant layer: typed Python calculators that read exact rates from SQLite and compose the premium, plus text-to-SQL lookups against the per-table information database. If it is a concept question, it is sent to the Retriever, which performs semantic search over the manual's prose. The language model then composes a final answer, citing manual pages and showing the exact rates used; a Verifier step checks the unit of each figure (percent vs per-mille vs franc amount) and that prose claims are grounded in retrieved passages.

```
                  +-----------------------------+
user question --> |      Manager / Router       |
                  | (classify: number vs prose) |
                  +--------------+--------------+
                    |                          |
           number / quote                concept / definition
                    |                          |
                    v                          v
      +-----------------------------+  +-----------------------------+
      |   Quant  (typed Python)     |  |   Retriever  (ChromaDB)     |
      |  pricing calculators +      |  |  semantic search over the   |
      |  read-only text-to-SQL      |  |  manual prose (embeddings)  |
      +-------------+---------------+  +-------------+---------------+
                    |                                |
                    v                                v
      +-----------------------------+  +-----------------------------+
      | SQLite rate tables          |  | corpus.md section chunks    |
      | assar.db / assar_info.db    |  | + page citations            |
      +-------------+---------------+  +-------------+---------------+
                     \                             /
                      v                           v
                  +------------------------------------+
                  |  LLM composes a cited answer;      |
                  |  Verifier checks units & sources   |
                  +------------------------------------+
```

The deliberate trade-off is determinism and auditability over end-to-end neural generation. A purely generative system would be simpler but impossible to trust for binding figures, because embeddings blur exact values and a model reading a rate out of retrieved text can round or transcribe it wrong. By confining the model to understanding the request, selecting a tool, and phrasing the result, the financially sensitive arithmetic stays under tested, reproducible control. A separate quote path bypasses the model entirely and calls the calculators directly, so pricing works even with no language-model key configured.

The language model is pluggable (Groq, Ollama, or Hugging Face via one OpenAI-compatible client). Embeddings always run locally. Premiums are produced by typed functions covered by 39 deterministic tests.

---

3. Data Design Choices

The system keeps two SQLite databases, both built from the same manual but shaped for different consumers. This is the central data decision.

The first, assar.db, uses four generic tables. A single rate table holds most per-category rates, namespaced by a scheme column (fire, public_liability, aviation, bond, pvt, and so on). Separate tables hold the two-dimensional transit commodity grids, the ordered discount and multiplier schedules, and per-product constants such as minimum premiums and mandatory excesses. This compact, normalized shape is convenient for the pricing calculators to join and iterate in code, and it is what the 39 tests pin.

The second, assar_info.db, inverts the design: one cleanly named SQL table per table in the manual, 45 in total. A question such as "what is the fire rate for a bank" maps naturally to a query against a fire_allied_perils table with a risk_category column, so a text-to-SQL agent does not need to know an internal namespacing convention. Descriptive table and column names are, in effect, the schema documentation the model reads to ground its query.

Three further choices shape the data. First, numbers were verified before being reused: the rate values were spot-checked cell by cell against the source PDF, and the information database reuses those verified values rather than re-transcribing them, confining new manual transcription to the large-risk registers. Second, labels in the information database are exact verbatim strings from the manual, including punctuation, so an agent's text matching lines up with the source wording; a build-time guard fails if any label list drifts out of alignment with its verified values. Third, units are made explicit: most columns carry their unit in the column name (rate_pct, rate_per_mille, insured_value_rwf), and a data_dictionary table documents the unit of every column, so a percentage is never mistaken for a per-mille rate, the easiest order-of-magnitude error in this manual.

---

4. Chunking Strategy

The prose corpus is chunked for retrieval with a structure-aware, section-based strategy rather than a fixed page or character window, because the manual is a long, structured document whose logical units are sections that often span page breaks.

The corpus builder extracts page text into a Markdown corpus, tagging each page so chunks can be cited. The chunker then detects heading runs, the manual's section titles such as "PRICING OF CONSEQUENTIAL LOSS" or "FULL VALUE BASIS", including titles that wrap onto a second line, and groups each section's body together even across page boundaries. Long sections are sub-split on sentence boundaries at roughly 1,600 characters with 200 characters of overlap. Every chunk is prefixed with its section title, which gives the embedding model context about what the chunk is, and carries its page anchor for citation plus the section name in metadata. Front-matter such as the cover page and table of contents is dropped, and stray PDF bullet glyphs are stripped.

The most important chunking decision is what is excluded: the numeric rate tables are not embedded at all. They live in SQLite and are served exactly. This removes the single biggest failure mode for a financial document, an embedding fuzzing a value like 0.3144 percent, and it is why the corpus holds only definitions, conditions, warranties, and guidance.

---

5. Retrieval Strategy

Concept questions are answered by retrieval-augmented generation over the prose corpus. Chunks are embedded locally with a sentence-transformers model (BAAI/bge-small-en-v1.5; a multilingual model is a drop-in for Kinyarwanda or French) and stored in a persistent ChromaDB collection using cosine similarity.

At query time the same model embeds the question, the top matches are fetched, and the language model composes an answer grounded in those passages with page citations. The retrieved section title and page travel with each result, so an answer can point the reader to the exact place in the manual.

The router is hybrid. Number and quote questions are routed to the Quant layer, which either calls a typed pricing calculator or runs a read-only SQL lookup against the information database; concept questions are routed to the retriever. Lookups are made robust with scheme-scoped fuzzy matching, so an approximate category such as "hotel" resolves to the valid key, and the dispatcher coerces string numbers and drops arguments a calculator does not accept. The model is restricted to general (non-life) business and declines motor, life, or medical questions rather than inventing a quote. The conversation is multi-turn: prior turns are passed back so a follow-up like "and for a bank?" keeps context.

---

6. Audit: SQL Tables Mapped to the Source Manual

The table below maps every table in the information database (assar_info.db) to the table or section it was built from in the ASSAR manual and the page(s) where that source appears, so the data can be verified against the PDF. SQL names are exactly as they appear in the database.

SQL TABLE	PAGE(S)	MANUAL TABLE / SOURCE
voluntary_deductible_discount	p.13	Schedule of Approved Discounts for Voluntary Deductible
special_perils	p.15	Special Perils and Respective Perils Premium Rate
short_period_scale	p.16	Scale of Rates for Short Period Insurance
fire_allied_perils	p.19-22	Fire and Allied Perils Insurance - Commercial/Administrative (Material Damage)
fire_private_dwellings	p.22	Fire Private Dwellings (All Perils Inclusive)
plate_glass	p.22	Pricing of Plate Glass Insurance
consequential_loss_basis	p.24	Rating for Consequential Loss - basis items (Gross Profit / Auditors Fees / Wages)
consequential_loss_indemnity_period	p.24	Indemnity Period Selected - Percentages of the Basis Rate
business_interruption_time_excess	p.24	Voluntary Time Excess under Business Interruption
burglary_full_value	p.28	Burglary & Theft - Full Value Basis minimum rates
first_loss_multiplier	p.28	Risks Insured on First Loss Basis - multipliers
bankers_blanket_bond	p.31	Bankers Blanket Bond - Description of Risk / Rate
directors_officers_liability	p.32	Directors and Officers Liability - Description

		of Risk / Rate
money_insurance	p.33-34	Money and Cash in Transit - rates (in safe, ATM, out of safe, custody)
money_annual_carryings	p.33	Money in Transit - Annual Carryings rate bands
goods_in_transit	p.36-38	Goods in Transit - commodity rate grid
transporters_liability	p.42-44	Transporters Liability - commodity rate grid
public_liability	p.46	Public Liability - Minimum Premium Rate
employers_liability	p.47	Employers' Liability - Minimum Premium Rate
school_liability	p.47-48	School Liability - premiums and indemnity limits
school_liability_short_period	p.48	Short Rates for School Liability
product_liability	p.49	Product Liability - Minimum Premium Rate
professional_indemnity	p.50	Professional Indemnity - Professional Classification / Rate
personal_accident_gpa	p.51	PA & GPA Risks Categories and Minimum Premium / Rates
personal_accident_short_period	p.52	Short Rates for Personal and Group Personal Accident
erection_all_risks	p.53-54	Erection All Risks (EAR) - Minimum Rate
machinery_breakdown	p.55-56	Machinery Breakdown - Minimum Rates
cpm_rates	p.57	Contractors Plant & Machinery (CPM) - Hazard Class x Plant Group
cpm_short_period	p.57-58	Short Period Rates under CPM
boilers_pressure_vessels	p.58	Boilers and Pressure Vessels - Material Damage / TPL
eear_computer_all_risks	p.58-59	Computer and Electric & Electronic All Risks (EEAR)
contractors_all_risks	p.62-63	Contractors' All Risks - Minimum Rate
aviation	p.64	Pricing of Aviation Risks - Description of Risk / Applicable Rate
marine_hull	p.66	Rates for Vessels / Marine Hull rating procedure
marine_hull_occupant_premiums	p.66	Premiums and Sums Insured for 1 Occupant (Bodily Injuries)
marine_cargo	p.67-69	Rates for Marine Cargo (ICC-A)
bonds_guarantees	p.70	Pricing of Bonds/Guarantees - Description of Bond / Rate
fidelity_guarantee	p.71	Pricing of Fidelity Guarantee - Description of Risk / Rate
pvt_political_violence_terrorism	p.72-73	Pricing of PVT Risks - Description of Risk / per-mille rate
large_risks_property	p.76-79	List of Large Risks Established in the Market - Property
large_risks_engineering	p.80	List of Large Risks - Engineering
large_risks_accident	p.80	List of Large Risks - Accident Classes
market_parameters	p.12-75	Derived: policy fee (p.12), FEA discount (p.16), stock declaration (p.29), commission (p.75)
minimum_premiums	multiple	Derived: minimum premiums across classes (money p.34, liabilities p.46-49, PA/GPA p.52, bonds p.71, fidelity p.72)
data_dictionary	n/a	Engine metadata - documents the unit of every column (not a manual table)

Total: 45 tables.

Audit notes. Pricing values are reused from the verified seed used by the calculators; the information database adds verbatim labels and the large-risk registers. Three tables are derived rather than transcribed from a single table:

market_parameters gathers scattered market-wide figures, minimum_premiums gathers per-class minimums, and data_dictionary is engine metadata documenting units. PVT rates are stored in per mille; every other class is percent.

---

7. Deployment and User Interface

The system runs as a Streamlit web application launched with a single command, backed by the two SQLite files and a local ChromaDB store. Setup installs the requirements, copies the environment template, and adds a language-model key (Groq by default; the key lives in a gitignored .env). The data is built in three steps: the pricing database from the transcribed tables, the information database (one table per manual table), and the prose corpus plus its vector store, after which the app is started. The embedding model is pre-warmed at startup so the first chat message is not a silent wait. The databases are committed so the project runs out of the box.

The interface is a wide layout with a themed banner and a sidebar that shows live status: the rate database and its row count, the information engine and its 45 tables, the language-model backend and whether a key is set, and whether the vector store is built. A standing caution reminds users to verify rates against the source manual before binding cover.

---

8. What It Does, Section by Section

Chat. The primary, client-facing surface is a multi-turn chat with conversation memory and a strip of product tiles showing the classes covered. A question is classified and routed; quote answers show a card with the product, the exact rate and unit, and the final premium, with a collapsible breakdown and a panel of cited manual passages. Because it remembers the conversation, follow-up questions work naturally. It can quote every class in the manual that has a calculator, twenty-one tools in all, and answers concept questions from the prose with page citations; out-of-scope questions are politely declined.

Get a Quote. A deterministic calculator with dropdowns for the main products. It bypasses the language model entirely and calls the typed calculators directly, so it always works and always returns exact figures with a full breakdown and the applicable excess. This is the reliable path for a precise quote.

Database. A transparency surface that lets a user browse and run read-only SQL against either database: the four-table pricing engine, or the 45-table information engine with its example queries and the data_dictionary of units. Results can be filtered, searched, and downloaded as CSV. Only SELECT and WITH queries are allowed, on a genuinely read-only connection.

Underneath all three, the pricing calculators read exact rates from SQLite and compose premiums with minimum-premium floors, mandatory excesses, short-period factors, and policy fees as the manual specifies, and they are covered by deterministic tests so a wrong cell or a broken composition is caught immediately. The whole system is designed to be deterministic where the numbers must be exact, fluent where the questions are open-ended, and auditable throughout. As always, the figures should be verified against the source manual before binding cover.

End of report.